C# Properties

Introduction to Properties

- In C#, properties are natural extension of data fields.
- They are usually known as 'smart fields' in C# community.
- Data encapsulation and hiding are the two fundamental characteristics of any object oriented programming language.
- By using various access modifiers like private, public, protected, internal etc…it is possible to control the accessibility of the class members.
- Usually inside a class, we declare a data field as private and will provide a set of public SET and GET methods to access the data fields.
- A C# property is composed using a get block and set block.
- The “Value” keyword represents the right-hand side of the assignment. “Value” is also an object.
- This is a good programming practice, since the data fields are not directly accessible out side the class. We must use the set/get methods to access the data fields.

```
using System;

class MyClass
{
  private int x;
  public void SetX(int i)
  {
        x = i;
  }
  public int GetX()
  {
        return x;
  }
}
class MyClient
{
  public static void Main()
  {
        MyClass MyVer = new MyClass();
        MyVer.SetX(10);
        int xVal = MyVer.GetX();
        Console.WriteLine(xVal);
  }
}
```

Note: Conventional programming method.

Properties Syntax

- C# provides a built in mechanism called properties to do the above.
- In C#, properties are defined using the property declaration syntax. The general form of declaring a property is as follows.
- <acces_modifier> <return_type> <property_name>

```
{
  get
  {
  }
  set
  {
  }
}
```

Last Program can be modified as:

```
class MyClass
{
    private int x;
    public int VisibleX
    {
     get
     {
          return x;
     }
     set
     {
          x = value;
     }
    }
}
```

```
using System;
class MyClass
{
    private static int x;
    public static int X
    {
      get
      {
            return x;
      }
      set
      {
            x = value;
      }
    }
}
class MyClient
{
    public static void Main()
    {
      MyClass.X = 10;
      int xVal = MyClass.X;
      Console.WriteLine(xVal);
    }

}
```

Advantages

For example,

```
//Conventional Programming method
Employee abc = new Employee();
abc.SetAge( abc.GetAge() + 1);

abc.Age++;
```

Advantages

For example Read Only Property,

```
public class employee
{

    // Assume this is assigned in the class constructor
    private string SocialSercurityNo;

    // A read only property
    public string SSN
    {
      get
      {
            return SocialSercurityNo;
      }
    }
```

Advantages

For example Static Property,

```
public class employee
{
    // Assume this is assigned in the class constructor
    private string static CompanyName;
    public string static Company
    {
      get
      {
            return CompanyName;
      }
      set
      {
            CompanyName = value;
      }
    }
    Public static void Main (string args[])
    {
       Employee.Comany = “Abc Ltd.”;
       Consol.WriteLine(Employee.Comany);
    }
```

Static Properties

- C# also supports static properties, which belongs to the class rather than to the objects of the class.
- All the rules applicable to a static member are applicable to static properties also.

Advantage

- **The main advantage over using a Property instead of a public field is,**

  with the property, you will get a chance to write few lines of code (if you want) in the get and set accessors. So, you can perform some validation or any other logic before returning any values or assigning to the private field.

Advantage

```
public class Car
{
      // private fields.
      private string color;
      // constructor public Car()
      {
       }
      public string Color
      {
      get
      {
            if ( color == "" )
               return "GREEN";
            else
               return color;
      }
```

Advantage

```
set
{
      if ( value == "" )
          thrown new Exception ("Wrong value.");
      else color = value;
}}}
```

- Here we are checking whether there is a valid value in the field 'color' before we return the value.
- If it is empty, we are getting a chance to return a default value 'Green'.
- In the set part, we are doing a validation to make sure we always assign a a valid value to our field.
- If someone assign an empty string to the 'Color' property, he will get an exception (error).

```
Car car = new Car();
car.Color = "";
```

Summary

No more public fields!

Always have private fields and write public properties as wrapper for them if required to expose them to outside the class.